

Grille d'autoévaluation - Semaine 5

420-W30-SF · Tell Don't Ask, SOLID, composition et Strategy · Automne 2026

Nom (facultatif)

Date de révision

Comment utiliser cette grille

Cochez une compétence seulement si vous pouvez l'expliquer et l'appliquer sans suivre une recette. Utilisez les cases non cochées pour planifier votre révision.

1. Tell, Don't Ask

Je suis capable de :

- ☐ repérer un client qui extrait des données pour décider ailleurs;
- ☐ nommer l'objet qui possède l'information nécessaire;
- ☐ remplacer une interrogation par une intention métier;
- ☐ replacer la validation près des données qu'elle protège;
- ☐ distinguer Tell, Don't Ask de la loi de Déméter;
- ☐ expliquer le risque de dupliquer une règle dans plusieurs clients.

Intention métier que je peux formuler

2. Principe ouvert-ferme

Je suis capable de :

- ☐ repérer une cascade conditionnelle qui grandit avec chaque variation;
- ☐ nommer le comportement stable et la variation réelle;
- ☐ définir un point d'extension ciblé;
- ☐ ajouter une implantation sans modifier les cas existants;
- ☐ expliquer pourquoi OCP ne demande pas une abstraction pour chaque détail;
- ☐ relier le point d'extension au besoin de ses clients.

Variation réelle qui justifie un point d'extension

3. Substitution de Liskov

Je suis capable de :

- ☐ formuler les attentes promises par un type de base;
- ☐ vérifier les entrées, les résultats et les invariants d'un sous-type;
- ☐ repérer une exception surprenante ajoutée par un sous-type;
- ☐ expliquer le problème carré-rectangle;
- ☐ proposer une abstraction plus honnête comme IForme;
- ☐ examiner l'immutabilité ou la composition comme corrections possibles;
- ☐ reconnaître un client obligé de tester le type concret.

Attente de substitution que je dois mieux justifier

4. Ségrégation des interfaces

Je suis capable de :

- ☐ repérer un client qui dépend d'une opération inutile;
- ☐ distinguer une interface cohérente d'une interface seulement courte;
- ☐ définir des contrats selon les besoins des clients;
- ☐ faire implanter plusieurs petits contrats par une même classe;
- ☐ expliquer pourquoi la longueur seule ne prouve pas une violation d'ISP.

Client et membre inutile que je peux identifier

5. Héritage ou composition

Je suis capable de :

- ☐ vérifier la substitution avant de proposer un héritage;
- ☐ expliquer pourquoi l'héritage ne sert pas seulement à réutiliser du code;
- ☐ choisir la composition pour un comportement remplaçable;
- ☐ choisir la composition lorsque plusieurs capacités se combinent;
- ☐ justifier le choix avec le contrat attendu par le client.

Relation dont je peux justifier le choix

6. Patron Strategy

Je suis capable de :

- ☐ repérer le comportement variable que le contexte doit déléguer;
- ☐ identifier le contexte, le contrat et les stratégies concrètes;
- ☐ placer le choix de la stratégie au point de composition;
- ☐ injecter la stratégie sans condition sur son type concret;
- ☐ relier Strategy à OCP, DIP et à la composition;
- ☐ distinguer le patron Strategy du seul mécanisme de polymorphisme;
- ☐ vérifier que la cascade n'a pas seulement été déplacée.

Rôles Strategy que je peux nommer dans un exemple

7. Défis de vérification

Sans consulter immédiatement les notes, je peux :

- ☐ replacer une règle dans l'objet qui possède les données;
- ☐ justifier un point d'extension avec une variation réelle;
- ☐ diagnostiquer une violation de LSP ou d'ISP;
- ☐ choisir entre héritage et composition;
- ☐ reconnaître les rôles de Strategy et situer la délégation;
- ☐ distinguer Strategy d'un simple usage du polymorphisme;
- ☐ relier Strategy à OCP, DIP, la composition et l'injection.

8. Mon plan de révision

Notion prioritaire et prochaine action concrète

Question à apporter en classe